

Оглавление

1 Аналитический раздел	6
1.1 Анализ предметной области	6
1.2 Сравнительный анализ существующих систем	6
1.3 Требования к проектируемой системе	7
1.3.1 Функциональные требования	7
1.3.2 Нефункциональные требования	8
1.3.3 Ограничения	8
1.4 Метрики эффективности и правила их расчёта	8
1.5 Сущности предметной области, роли пользователей и варианты использования	10
1.5.1 Сущности и связи	10
1.5.2 Роли пользователей и варианты использования	11
1.6 Выбор модели данных и архитектуры хранилища	12
1.6.1 Выбор логической модели данных	12
1.6.2 Выбор физической архитектуры (OLTP vs OLAP)	13
2 Конструкторский раздел	15
2.1 Логическое проектирование базы данных	15
2.1.1 Соответствие нормальным формам	20
2.1.2 Проектирование представлений	20
2.1.3 Проектирование индексов	21
2.2 Проектирование ограничений целостности	21
2.3 Проектирование алгоритмов обработки данных	22
2.3.1 Алгоритмы расчёта метрик игроков	22
2.3.2 Алгоритм агрегирования командной статистики	22
2.3.3 Алгоритм массового обновления агрегатов	23
2.4 Проектирование ролевой модели доступа	23
2.5 Проектирование программного обеспечения и кэширования	24
2.5.1 Архитектура приложения	24
2.5.2 Спецификация REST API	24
2.5.3 Кэширование результатов запросов	26
3 Технологический раздел	28

3.1	Выбор и обоснование средств реализации	28
3.2	Физическая реализация базы данных	29
3.2.1	Создание таблиц и ограничений	29
3.2.2	Реализация индексов	30
3.2.3	Загрузка тестовых данных	30
3.3	Реализация логики предметной области в СУБД	31
3.3.1	Хранимые функции расчёта метрик	31
3.3.2	Триггеры логической целостности	32
3.4	Реализация ролевой модели доступа	32
3.5	Реализация программного обеспечения и интерфейса	33
3.5.1	Слой маршрутизации и взаимодействия с БД (Backend)	33
3.5.2	Пользовательский интерфейс (Frontend)	34
3.6	Тестирование и верификация аналитических данных	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		37
ПРИЛОЖЕНИЯ		39
	Приложение А. Скрипты создания структуры базы данных	39
	Приложение Б. Скрипты создания функций и процедур	39
	Приложение В. Исходный код механизма кэширования	40

1 Аналитический раздел

1.1 Анализ предметной области

Национальная баскетбольная ассоциация (NBA) является источником одного из наиболее детализированных массивов спортивной статистики среди крупнейших мировых профессиональных лиг [1]. Для каждого матча фиксируются итоговый счёт, составы команд и числовые показатели по каждому игроку: очки, подборы, передачи, блок-шоты, перехваты и иные. На этой основе можно строить рейтинги, сопоставлять сезоны и выявлять тенденции развития игры.

В современной баскетбольной аналитике для комплексной оценки результативности применяются продвинутое метрики, стандартизированные в работах ведущих спортивных статистиков [2]. Расчёт таких показателей как истинный процент бросков TS%, эффективный процент с игры eFG%, доля владений в атаке USG%, рейтинг эффективности игрока PER и показатель вклада игрока BPM требует хранения непротиворечивых первичных данных по матчам, их агрегирования по сезонам и применения фиксированных правил вычисления.

Обработка спортивной статистики требует строгого контроля над целостностью данных: сумма очков игроков должна совпадать с командным счётом, агрегаты по сезонам должны вычисляться из одних и тех же первичных фактов, а параллельный доступ не должен нарушать консистентность записей. Это определяет необходимость специализированного хранилища с поддержкой транзакционной целостности и декларативной валидации бизнес-правил.

1.2 Сравнительный анализ существующих систем

Перед проектированием собственного решения проведём анализ функциональных возможностей наиболее распространённых публичных систем сбора и анализа статистики NBA: официального портала NBA Stats [1], справочной платформы Basketball-Reference [4] и аналитического раздела ESPN [5].

Таблица 1.1 — Сравнительный анализ существующих систем по функциональным возможностям

Критерий	NBA.com Stats	Basketball-Reference	ESPN Stats
Просмотр статистики матчей	+	+	+
Фильтрация по сезону, позиции, команде	частично	+	частично
Расчёт TS%, eFG%, PER, BPM	+ (формулы скрыты)	+ (формулы открыты [7, 8])	частично
Доступ к сырым данным через API (произвольные запросы)	–	–	–
Интерактивное сравнение стат. профилей игроков	частично	–	+
Ролевое разграничение доступа	–	–	–
Локальное хранение, воспроизводимость расчётов	–	–	–

Анализ показывает, что NBA.com Stats и ESPN предоставляют данные через готовый пользовательский интерфейс с непрозрачными формулами расчёта метрик. Basketball-Reference публикует описание своих формул [7, 8], однако не предоставляет открытого API для произвольных аналитических запросов. Ни одна из систем не поддерживает ролевого разграничения доступа и воспроизводимого локального хранения данных. Выявленные ограничения определяют требования к проектируемой системе, сформулированные в следующем разделе.

1.3 Требования к проектируемой системе

1.3.1 Функциональные требования

- 1) хранение справочных данных: игровые позиции, сезоны, клубы NBA;
- 2) хранение профилей игроков: персональные и антропометрические данные, позиция, сведения о драфте;
- 3) хранение данных о матчах: сезон, команды-участники, дата, итоговый счёт, статус;
- 4) хранение детализированной статистики игрока в матче: очки, подборы, передачи, броски с игры и штрафные, потери, фолы, время на площадке;
- 5) хранение агрегированной статистики игрока за сезон: средние значения, процентные показатели, расчётные метрики;
- 6) хранение истории выступлений игрока за клубы по сезонам;
- 7) расчёт командной статистики в рамках матча и сезона на основе индивидуальных показателей игроков;
- 8) автоматизированный расчёт метрик эффективности (eFG%, TS%, USG%, PER, BPM) на основе первичных фактов матчей;
- 9) предоставление сводных рейтингов игроков и турнирных таблиц команд.

1.3.2 Нефункциональные требования

- 1) производительность: время отклика на аналитические запросы чтения (агрегация за сезон) не должно превышать 200 мс;
- 2) объём данных: система должна выдерживать нагрузку не менее 100 000 записей в таблице фактов (соответствует статистике за 5 регулярных сезонов);
- 3) целостность: база данных должна соответствовать требованиям ACID, обеспечивая ссылочную и логическую целостность на уровне СУБД;
- 4) безопасность: ролевое разграничение доступа (администратор, аналитик, читатель), реализованное средствами СУБД на уровне таблиц и представлений;
- 5) масштабируемость чтения: система должна обеспечивать кэширование результатов повторяющихся аналитических запросов для снижения нагрузки на дисковую подсистему при конкурентном доступе.

1.3.3 Ограничения

- 1) глубина хранения данных ограничена 5 регулярными сезонами (с 2019/20 по 2023/24);
- 2) пользовательский интерфейс предназначен для просмотра и анализа данных, а не для ввода первичной статистики в режиме реального времени.

1.4 Метрики эффективности и правила их расчёта

Обозначения: PTS — очки; FGM, FGA — попадания и попытки с игры; FG3M — трёхочковые; FTM, FTA — штрафные; TOV — потери; TRB — подборы; AST — передачи; MP — минуты. Префикс Tm обозначает командный агрегат по матчам с участием игрока: TmFGA, TmFTA, TmTOV; TmMP — суммарное количество минут, отыгранных всеми игроками команды в матчах, учитываемых при расчёте показателя данного игрока.

Командная статистика не выделяется в отдельную сущность: она является производной от индивидуальных показателей и вычисляется динамически суммированием по всем игрокам команды в конкретном матче.

Эффективный процент с игры, eFG%. Метрика корректирует ценность трёхочкового броска. Согласно методологии NBA [9], формула имеет вид:

$$eFG\% = \frac{FGM + 0,5 \cdot FG3M}{FGA} \quad (1.1)$$

Истинный процент бросков, TS%. Показатель объединяет броски с игры и штрафные броски в единую метрику эффективности. Множитель 0,44 в знаменателе является эмпирически выведенным коэффициентом Дина Оливера, учитывающим долю штрафных бросков,

завершающих владение [10]:

$$TS\% = \frac{PTS}{2 \cdot (FGA + 0,44 \cdot FTA)} \quad (1.2)$$

Доля владений в атаке, USG%. Оценивает процент командных владений, которые завершает конкретный игрок, находясь на площадке [2]. В базе данных показатель хранится как доля $[0; 1]$; в интерфейсе отображается в процентах ($\times 100$). Формула:

$$USG = \frac{(FGA + 0,44 \cdot FTA + TOV) \cdot (TmMP/5)}{MP \cdot (TmFGA + 0,44 \cdot TmFTA + TmTOV)} \quad (1.3)$$

Деление $TmMP$ на 5 нормирует командное время с учётом того, что на площадке одновременно находятся пять игроков.

Рейтинг эффективности игрока, PER. Оригинальная формула разработана Джоном Холлинджером [6]. В настоящей работе применяется её упрощённая версия без позиционных поправок, методология которой описана на портале Basketball-Reference [7]. Подборы $TRB = REB_OFF + REB_DEF$. Ненормализованный вклад приводится к шкале «за 48 минут», затем масштабируется к среднему по лиге, равному 15:

$$uPER_i = \left[PTS + TRB + AST + STL + BLK - TOV - PF - (FGA - FGM) - 0,44 \cdot (FTA - FTM) \right] \cdot \frac{48}{MP_i} \quad (1.4)$$

$$lg_aPER = AVG(uPER_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.5)$$

$$PER_i = uPER_i \cdot \frac{15}{lg_aPER} \quad (1.6)$$

Минимальный порог $MP_i \geq 50$ минут за сезон; для расчёта lg_aPER учитываются игроки с $MP_i > 100$ минут.

Вклад игрока (Box Plus/Minus, BPM). Метрика разработана Дэниелом Майерсом [8]. В системе используется классическая линейная аппроксимация первой версии алгоритма (BPM 1.0) с нормировкой показателей на 100 минут ($X_{100} = X/MP \cdot 100$) и учётом перехватов, блоков и фолов. Итоговая формула:

$$\begin{aligned} raw_i = & 0,123 \cdot PTS_{100} + 0,234 \cdot TRB_{100} + 0,689 \cdot AST_{100} + 0,445 \cdot STL_{100} \\ & + 0,407 \cdot BLK_{100} - 0,605 \cdot TOV_{100} - 0,086 \cdot PF_{100} \end{aligned} \quad (1.7)$$

$$BPM_i = raw_i - AVG(raw_i) \quad \text{по игрокам с } MP_i > 100 \quad (1.8)$$

Среднее BPM по лиге после вычитания среднего равно 0. Минимальный порог для расчёта показателя игрока — $MP_i \geq 50$ минут за сезон.

1.5 Сущности предметной области, роли пользователей и варианты использования

1.5.1 Сущности и связи

По результатам анализа предметной области выделены восемь основных сущностей.

- 1) позиция — справочник игровых позиций в номенклатуре NBA;
- 2) сезон — игровой год, его календарные границы и тип турнира;
- 3) команда — клуб NBA: конференция, дивизион, арена;
- 4) игрок — персональные и антропометрические данные, позиция, сведения о драфте;
- 5) матч — сезон, команды-участники, дата, счёт, статус;
- 6) статистика игрока в матче — числовые показатели конкретного игрока в конкретной игре; основная таблица фактов предметной области;
- 7) статистика игрока за сезон — агрегаты и расчётные метрики по тройке «игрок – сезон – команда»;
- 8) история выступлений — связь «игрок – команда – сезон» с датами и типом контракта.

Связь «многие-ко-многим» между игроками и матчами реализуется через сущность «Статистика игрока в матче»: один игрок участвует во многих матчах, в одном матче фиксируется статистика многих игроков [3, 11].

Связь «Проходит в» на ER-диаграмме является тернарной: она одновременно связывает сущности «Матч», «Команда» и «Сезон», поскольку матч всегда привязан к конкретному сезону и двум командам-участницам [11].

Концептуальная ER-диаграмма в нотации Чена приведена на рисунке 1.1; на ней отражены сущности предметной области, бинарные и тернарные связи.

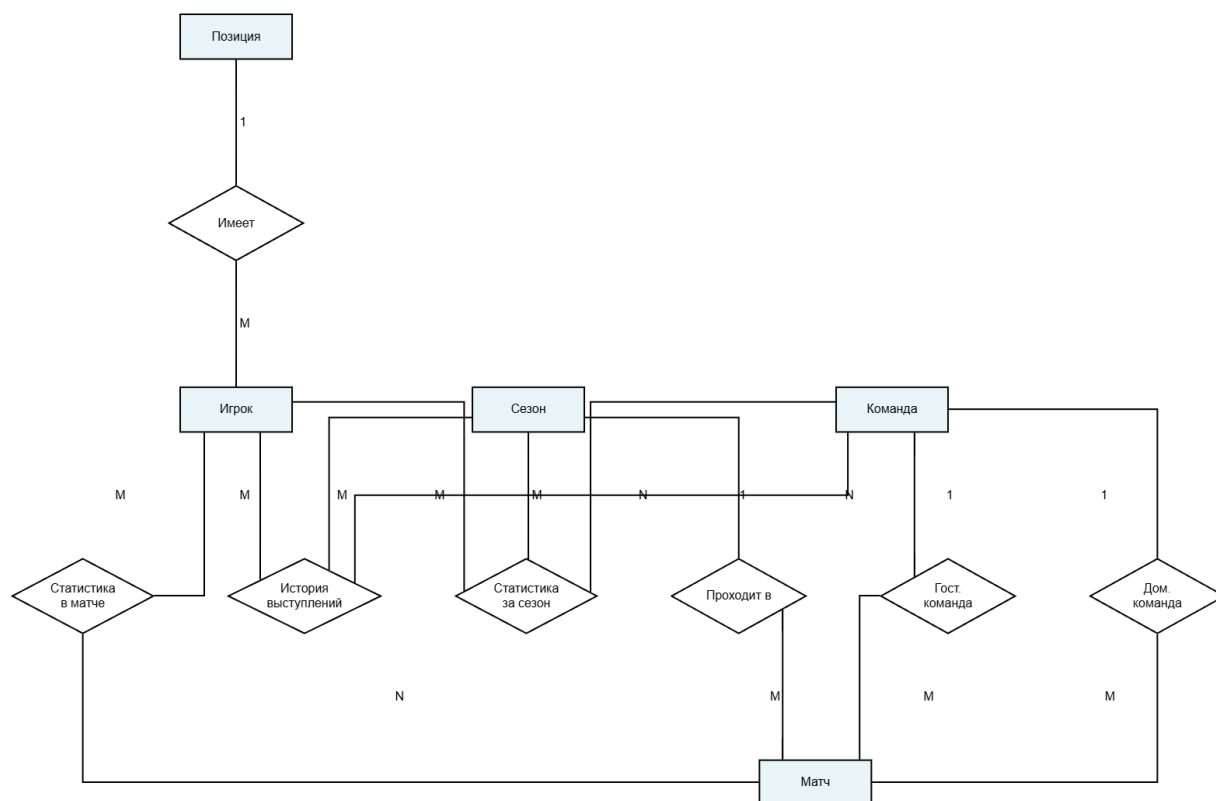


Рисунок 1.1 — ER-диаграмма в нотации Чена

1.5.2 Роли пользователей и варианты использования

Система предусматривает три роли пользователей.

- 1) администратор — полное управление структурой и содержимым хранилища: создание и изменение схемы, загрузка и сопровождение данных, резервное копирование;
- 2) аналитик — чтение данных и запуск расчётных процедур для получения производных показателей;
- 3) читатель — доступ к агрегированным данным через представления; изменение содержимого базовых таблиц недоступно.

Соответствие ролей и допустимых операций задаётся тремя вариантами использования.

- 1) просмотр статистики — получение сведений о матчах, игроках и командах, агрегированных показателях и рейтингах; доступно всем ролям;
- 2) расчёт метрик — вычисление продвинутых показателей (PER, BPM, eFG%, TS%, USG%) на основе первичных данных; доступно аналитику и администратору;
- 3) управление данными — загрузка данных о матчах, обновление справочников, изменение схемы; доступно только администратору.

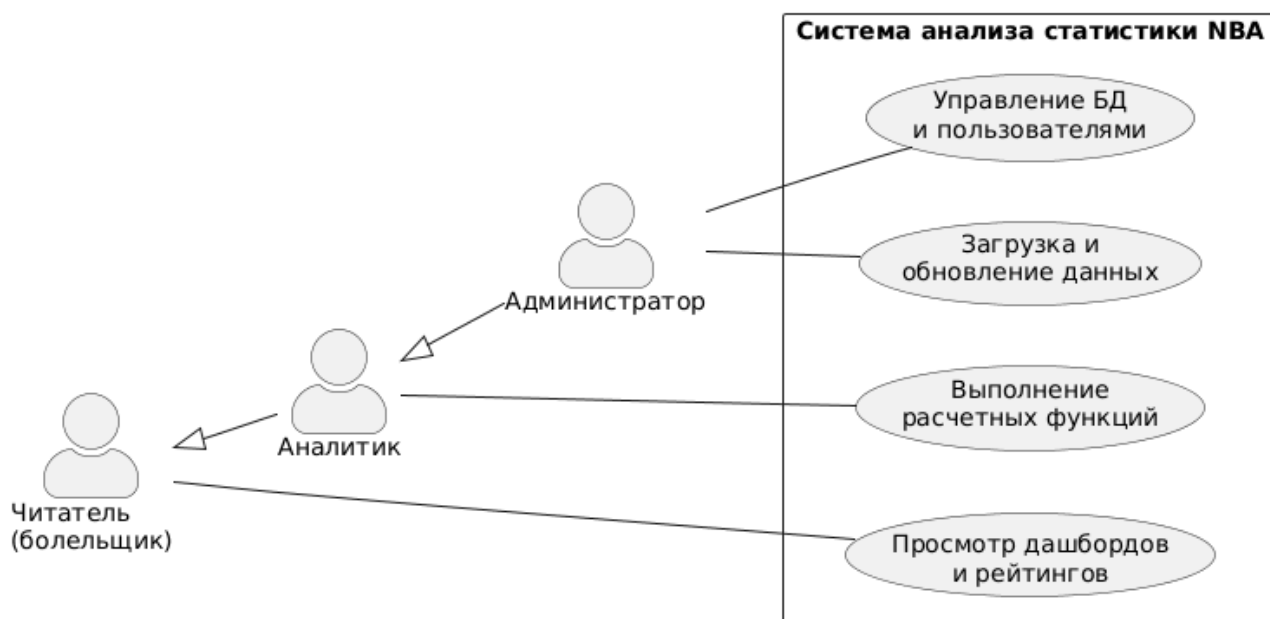


Рисунок 1.2 — Диаграмма вариантов использования

1.6 Выбор модели данных и архитектуры хранилища

Для выбора оптимального решения разделим задачу на два этапа: выбор логической модели данных и выбор физической архитектуры (профиля нагрузки) [12, с. 42].

1.6.1 Выбор логической модели данных

Рассмотрим основные логические модели данных для предметной области спортивной статистики.

Графовая модель данных (Neo4j). В данной модели данные представлены в виде узлов и рёбер (связей). Она оптимальна для обхода сложных социальных связей. Однако для баскетбольной статистики, где доминируют математические агрегации (вычисление средних, сумм, процентов) по сотням тысяч фактов, графовые СУБД демонстрируют крайне низкую производительность [12, с. 60].

Документно-ориентированная модель (MongoDB). Хранит данные в виде независимых иерархических структур (например, JSON). Как отмечают П. Садаладж и М. Фаулер [13, с. 35], такие базы данных оптимальны для агрегированных объектов без жёсткой схемы (schema-less). Сырая статистика матча (Box Score) естественно ложится в документ, однако агрегация данных по нескольким сезонам и связывание игроков с историей команд потребует сложных программных JOIN-операций, которые в документной модели неэффективны.

Реляционная модель данных. Представляет данные в виде двумерных таблиц с жёстко заданной схемой [3, с. 150]. Спортивная статистика обладает строгой, редко меняющейся структурой (игрок всегда имеет очки, подборы, фолы). Реляционная алгебра обеспечивает мощный математический аппарат для многомерных агрегаций, а сама модель (schema-on-write) предотвращает попадание аномальных данных (например, отрицательных очков) за счёт ссылочной

целостности и табличных ограничений.

Таблица 1.2 — Сравнительная характеристика логических моделей данных

Критерий	Графовая	Документная	Реляционная
Управление схемой	Отсутствует / Гибкая	Гибкая (schema-on-read)	Строгая (schema-on-write)
Математические агрегации фактов	Низкая эффективность	Средняя (через пайплайны)	Высокая (нативная оптимизация)
Связывание сущностей (JOIN)	Обход по ссылкам (быстро)	Затруднено (денормализация)	Нативная поддержка (внешние ключи)
Целостность фактов	На уровне приложения	Поддержка JSON Schema	Строгие констрейнты таблиц

На основе анализа логических моделей выбрана **реляционная модель**, так как данные строго структурированы и требуют сложных агрегаций с объединением справочников.

1.6.2 Выбор физической архитектуры (OLTP vs OLAP)

После выбора реляционной модели необходимо определить физический способ хранения данных: по строкам (Row-oriented) или по столбцам (Column-oriented).

Колоночные СУБД (ClickHouse). В данной архитектуре данные физически хранятся по колонкам, что многократно ускоряет агрегирующие аналитические запросы над миллионами записей (OLAP-профиль) [14]. Однако колоночные системы плохо справляются с точечными транзакционными обновлениями данных (UPDATE/DELETE).

Строковые транзакционные СУБД (PostgreSQL). Ориентированы на смешанную нагрузку (HTAP). Движок хранения гарантирует строгую согласованность данных (ACID-транзакции) на уровне физических страниц [15, с. 510].

Рассматриваемая система предполагает смешанный профиль: процедура обновления сезонной статистики выполняет ресурсоёмкие транзакционные операции слияния (UPSERT) для сотен записей. Также требуется ретроспективная корректировка протоколов матчей (точечные UPDATE). При целевом объёме таблицы фактов порядка 200 000 строк (5 сезонов) классическая строковая реляционная СУБД способна выполнять аналитические запросы без существенных задержек с помощью B-tree индексов. Отставание в скорости чтения сводных таблиц будет полностью нивелировано in-memoу кэшированием в Redis [16]. Использование колоночной СУБД при таком масштабе является архитектурной избыточностью.

Таким образом, для реализации хранилища целесообразно использовать реляционную СУБД со строковой физической архитектурой, дополненную in-memoу кэшированием.

Вывод

Сформулированы функциональные и нефункциональные требования и ограничения; определён перечень метрик и правила их вычисления. Выделены восемь сущностей, установлены связи «многие-ко-многим» через таблицы фактов, описаны три пользовательские роли.

По результатам сравнения логических и физических моделей данных для проектируемой системы выбрана реляционная модель со строковым хранением. В отличие от документоориентированных и графовых моделей, она позволяет эффективно выполнять многомерные агрегации фактов, а в отличие от колоночных систем — обеспечивает полную транзакционную целостность (ACID) при точечных и множественных обновлениях связанных таблиц.

Сравнительный анализ существующих систем (NBA.com Stats, Basketball-Reference, ESPN Stats) подтвердил отсутствие у них произвольных аналитических запросов, управляемого ролевого доступа и воспроизводимого локального хранения данных, что обуславливает актуальность и целесообразность разработки специализированной базы данных. Диаграмма вариантов использования и ER-диаграмма приведены на рисунках 1.2 и 1.1.

2 Конструкторский раздел

2.1 Логическое проектирование базы данных

Инфологическая модель трансформирована в реляционную схему из восьми таблиц. Справочник `seasons` использует суррогатный ключ `season_id` и метку `label` (формат «ГГГГ-ГГ»). Сборка фактов осуществляется в таблице `game_player_stats`, сезонные агрегаты хранятся в таблице `player_season_stats` в виде денормализованной витрины данных для минимизации времени отклика. Сводная схема связей между таблицами приведена на рисунке 2.1.

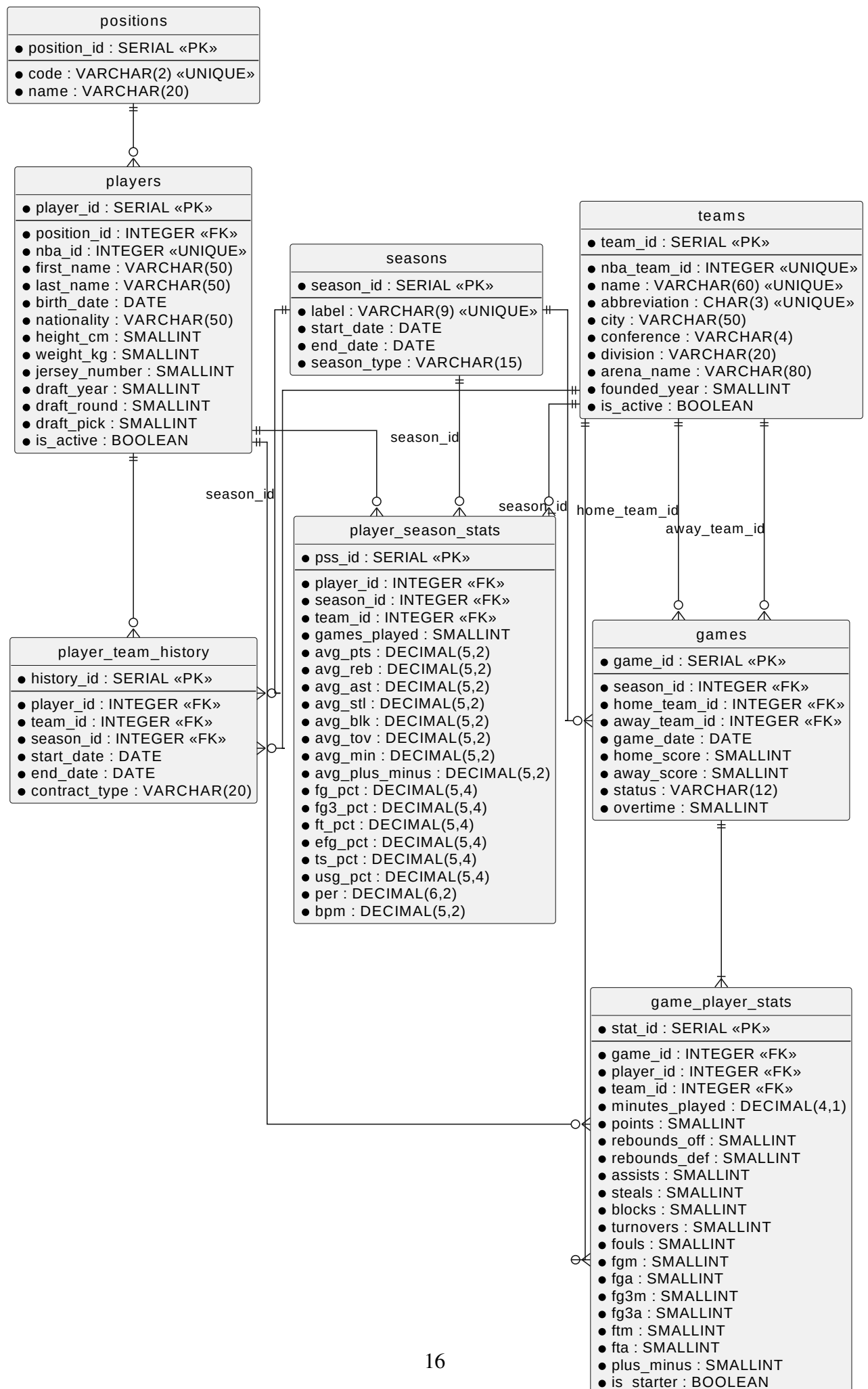


Таблица 2.1 — Таблица positions (справочник позиций)

Атрибут	Тип	Ключ / назначение
position_id	SERIAL	PK
code	VARCHAR(2)	UNIQUE; код PG, SG, SF, PF, C
name	VARCHAR(20)	полное название позиции

Таблица 2.2 — Таблица seasons (сезоны)

Атрибут	Тип	Ключ / назначение
season_id	SERIAL	PK
label	VARCHAR(9)	UNIQUE; метка сезона (напр. 2023-24)
start_date	DATE	дата начала сезона
end_date	DATE	дата окончания сезона
season_type	VARCHAR(15)	Regular, Playoff, Preseason

Таблица 2.3 — Таблица teams (команды)

Атрибут	Тип	Ключ / назначение
team_id	SERIAL	PK
nba_team_id	INTEGER	UNIQUE; id на NBA.com
name	VARCHAR(60)	UNIQUE; полное название
abbreviation	CHAR(3)	UNIQUE; аббревиатура
city	VARCHAR(50)	город базирования
conference	VARCHAR(4)	CHECK IN ('East','West')
division	VARCHAR(20)	дивизион NBA
arena_name	VARCHAR(80)	домашняя арена
founded_year	SMALLINT	год основания
is_active	BOOLEAN	признак активной команды

Таблица 2.4 — Таблица players (игроки)

Атрибут	Тип	Ключ / назначение
player_id	SERIAL	PK
nba_id	INTEGER	UNIQUE; id на NBA.com
first_name	VARCHAR(50)	имя
last_name	VARCHAR(50)	фамилия
birth_date	DATE	дата рождения
nationality	VARCHAR(50)	гражданство
height_cm	SMALLINT	рост (см)
weight_kg	SMALLINT	вес (кг)
position_id	INTEGER	FK → positions
jersey_number	SMALLINT	номер на майке (0–99)
is_active	BOOLEAN	действующий игрок
draft_year	SMALLINT	год драфта
draft_round	SMALLINT	раунд драфта (1 или 2)
draft_pick	SMALLINT	номер выбора на драфте

Таблица 2.5 — Таблица games (матчи)

Атрибут	Тип	Ключ / назначение
game_id	SERIAL	PK
season_id	INTEGER	FK → seasons
home_team_id	INTEGER	FK → teams; команда-хозяин
away_team_id	INTEGER	FK → teams; команда-гость
game_date	DATE	дата матча
home_score	SMALLINT	счёт хозяев
away_score	SMALLINT	счёт гостей
status	VARCHAR(12)	Finished, Scheduled, Postponed
overtime	SMALLINT	число овертаймов (0–6)

Таблица 2.6 — Таблица game_player_stats (факты матча)

Атрибут	Тип	Ключ / назначение
stat_id	SERIAL	PK
game_id	INTEGER	FK → games, ON DELETE CASCADE
player_id	INTEGER	FK → players, ON DELETE RESTRICT
team_id	INTEGER	FK → teams, ON DELETE RESTRICT
minutes_played	DECIMAL(4,1)	минуты на площадке
points	SMALLINT	очки (CHECK ≥ 0)
rebounds_off, rebounds_def	SMALLINT	подборы в нападении и защите
assists, steals, blocks	SMALLINT	передачи, перехваты, блоки
turnovers, fouls	SMALLINT	потери и фолы
fgm, fga, fg3m, fg3a, ftm, fta	SMALLINT	попадания и попытки бросков
plus_minus	SMALLINT	показатель плюс-минус
is_starter	BOOLEAN	признак стартового состава
UNIQUE	—	(game_id, player_id) — гарантирует единственность записи игрока в матче

В таблице game_player_stats пара полей game_id и player_id закреплена ограничением UNIQUE. Это исключает дублирование строк статистики одного игрока в одном матче.

Таблица 2.7 — Таблица player_season_stats (агрегаты за сезон)

Атрибут	Тип	Ключ / назначение
pss_id	SERIAL	PK
player_id	INTEGER	FK → players; часть UNIQUE
season_id	INTEGER	FK → seasons; часть UNIQUE
team_id	INTEGER	FK → teams; часть UNIQUE
games_played	SMALLINT	число игр в сезоне
avg_pts, avg_reb, avg_ast	DECIMAL(5,2)	средние за матч (очки, подборы, передачи)
avg_stl, avg_blk, avg_tov, avg_min	DECIMAL(5,2)	перехваты, блоки, потери, минуты
fg_pct, fg3_pct, ft_pct	DECIMAL(5,4)	проценты попаданий (дробь ∈ [0, 1])
avg_plus_minus	DECIMAL(5,2)	средний плюс-минус
efg_pct, ts_pct, usg_pct	DECIMAL(5,4)	eFG%, TS%, USG (доля 0–1; на UI × 100)
per	DECIMAL(6,2)	PER
bpm	DECIMAL(5,2)	BPM

Таблица 2.8 — Таблица `player_team_history` (история выступлений)

Атрибут	Тип	Ключ / назначение
<code>history_id</code>	SERIAL	PK
<code>player_id</code>	INTEGER	FK → <code>players</code> ; часть UNIQUE
<code>team_id</code>	INTEGER	FK → <code>teams</code>
<code>season_id</code>	INTEGER	FK → <code>seasons</code>
<code>start_date</code>	DATE	дата начала выступления
<code>end_date</code>	DATE	дата окончания (NULL — по сей день)
<code>contract_type</code>	VARCHAR(20)	Standard, Two-Way, 10-Day и др.

2.1.1 Соответствие нормальным формам

Схема нормализована до третьей нормальной формы (3НФ). Ниже для всех восьми таблиц проверяется выполнение первой (1НФ), второй (2НФ) и третьей (3НФ) нормальных форм [15, с. 390].

Первая нормальная форма (1НФ). В каждом отношении атрибуты атомарны: повторяющихся групп и многозначных полей нет. Статистика игрока в матче хранится отдельной строкой в `game_player_stats`, а не списком показателей в одной ячейке; справочники (`positions`, `teams`) и сущности (`players`, `games`) разнесены по отдельным таблицам.

Вторая нормальная форма (2НФ). Отношение в 2НФ, если оно в 1НФ и каждый неключевой атрибут полностью зависит от всего составного ключа. У таблиц `positions`, `seasons`, `teams`, `players`, `games` первичный ключ простой (`*_id`), поэтому частичных зависимостей быть не может. В `game_player_stats` показатели зависят от пары `game_id` и `player_id`, а не от каждого поля по отдельности. Аналогично в `player_season_stats` агрегаты зависят от составного ключа: `player_id`, `season_id`, `team_id`. В `player_team_history` атрибут `contract_type` относится к конкретному периоду игрока в команде и зависит от составного ключа записи.

Третья нормальная форма (3НФ). Отношение в 3НФ, если оно в 2НФ и для каждой нетривиальной зависимости $X \rightarrow A$ либо X — суперключ, либо A входит в потенциальный ключ. В `teams` поля `city` и `conference` описывают команду напрямую и зависят только от `team_id`. В `players` ФИО и физические параметры зависят только от `player_id`, а не от `team_id` (команда задаётся через `player_team_history`). Совпадение данных `player_season_stats` с агрегатами из `game_player_stats` — избыточность между таблицами (витрина для быстрых выборов), а не нарушение 3НФ внутри одного отношения.

Таким образом, все таблицы схемы находятся в 1НФ, 2НФ и 3НФ.

2.1.2 Проектирование представлений

Для обеспечения доступа к данным без прямого обращения к базовым таблицам спроектированы четыре представления:

- 1) `v_player_rankings` — объединяет `player_season_stats` и `players`; предоставляет

- сводный рейтинг игроков за сезон по очкам, подборам, передачам и расчётным метрикам;
- 2) `v_team_stats` — агрегирует данные `game_player_stats` в разрезе команды и сезона; реализует функциональное требование расчёта командной статистики без отдельной таблицы фактов;
 - 3) `v_team_standings` — вычисляет турнирное положение команд (число побед и поражений, процент побед) на основе завершённых матчей;
 - 4) `v_top_players` — краткая сводка лидеров по PER для роли reader (подмножество `v_player_rankings`).

2.1.3 Проектирование индексов

Для обеспечения производительности аналитических запросов в соответствии с нефункциональными требованиями (раздел 1.3.2) спроектированы следующие индексы [17]:

- 1) `idx_gps_player_id` ON `game_player_stats(player_id)` — ускоряет выборку всех выступлений игрока для агрегации сезонной статистики;
- 2) `idx_gps_team_game` ON `game_player_stats(team_id, game_id)` — поддерживает агрегацию командной статистики в представлении `v_team_stats`;
- 3) `idx_games_season_id` ON `games(season_id)` — ускоряет фильтрацию матчей по сезону;
- 4) `idx_games_date` ON `games(game_date)` — обеспечивает диапазонную фильтрацию по дате проведения матча;
- 5) `idx_pss_season` ON `player_season_stats(season_id)` — ускоряет выборку рейтингов за конкретный сезон;
- 6) `idx_players_name` ON `players(last_name, first_name)` — поддерживает поиск игрока по фамилии.

Ограничение `UNIQUE` на `(game_id, player_id)` в таблице `game_player_stats` автоматически создаёт B-tree индекс, покрывающий запросы поиска по паре «матч – игрок». Все индексы реализованы как B-tree — стандартная структура PostgreSQL, оптимальная для равенства и диапазонных предикатов в аналитических запросах.

2.2 Проектирование ограничений целостности

Помимо стандартных ограничений первичных (PK) и внешних (FK) ключей, в СУБД реализована логика защиты бизнес-правил.

Внешние ключи и каскадное удаление. При удалении матча из `games` связанные строки `game_player_stats` удаляются каскадно (`ON DELETE CASCADE`). Для справочников игроков и команд применяется стратегия `ON DELETE RESTRICT`: физическое удаление запрещено во избежание потери исторической статистики; вместо этого используется логическое удаление посредством флага `is_active`.

ЧЕКС-ограничения. Обеспечивают согласованность бросков: $fg3m \leq fgm$, $fgm \leq fga$;

диапазон фолов от 0 до 6. В games поля home_team_id и away_team_id различаются. В player_team_history выполняется $\text{start_date} \leq \text{end_date}$.

Бизнес-триггеры. На таблице game_player_stats реализованы два триггера. Триггер trg_check_team_score (AFTER INSERT OR UPDATE, FOR EACH ROW) валидирует равенство суммы очков игроков ($\sum \text{points}$) командному счёту (home_score или away_score) для завершённых матчей. Триггер after_game_player_stats_insert (AFTER INSERT, FOR EACH STATEMENT) вызывает update_season_stats для пересчёта витрины. Ограничение по счёту применимо к полностью обработанным данным: игроки, не принявшие участие в матче, в таблицу фактов game_player_stats они не вносятся; загружаемые источники должны обеспечивать атрибуцию всех очков конкретным игрокам.

2.3 Проектирование алгоритмов обработки данных

2.3.1 Алгоритмы расчёта метрик игроков

Функции расчёта метрик выполняются на стороне СУБД. При проектировании алгоритма учтено деление на ноль: если знаменатель равен нулю, функция возвращает NULL, что позволяет агрегатным функциям AVG() корректно исключить таких игроков из расчёта командной и лиговой статистики без внесения нулевых искажений.

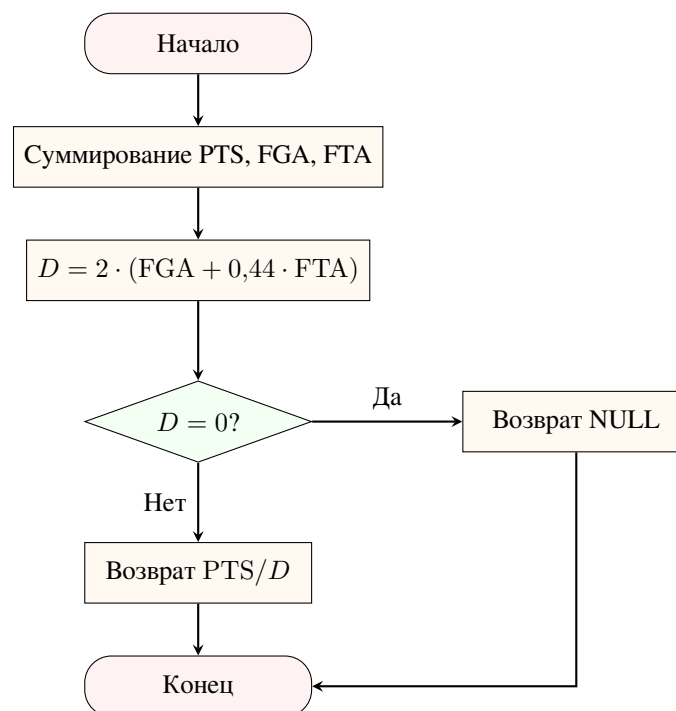


Рисунок 2.2 — Алгоритм функции calculate_pts

2.3.2 Алгоритм агрегирования командной статистики

Командная статистика не хранится в отдельной таблице: в соответствии с проектным решением она вычисляется динамически представлением v_team_stats. Для каждой пары «ко-

манда – сезон» представление суммирует показатели всех игроков из `game_player_stats` по матчам, в которых данная команда выступала в качестве хозяев или гостей, и делит накопленные суммы на число матчей. Принадлежность строки статистики к хозяевам или гостям определяется через JOIN с таблицей `games` по полям `home_team_id` и `away_team_id`. Результирующий набор содержит средние очки, подборы, передачи, потери и эффективные показатели команды за матч.

2.3.3 Алгоритм массового обновления агрегатов

Процедура `update_season_stats` пересчитывает агрегаты для указанного сезона. Алгоритм спроектирован с отказом от курсоров и построчного обхода. Применяется множественная операция с использованием конструкции `INSERT ... ON CONFLICT DO UPDATE`. Это позволяет СУБД задействовать внутренний оптимизатор и многократно сократить время пересчёта. Атомарность операции UPSERT на уровне СУБД предотвращает состояния гонки при параллельном пересчёте.

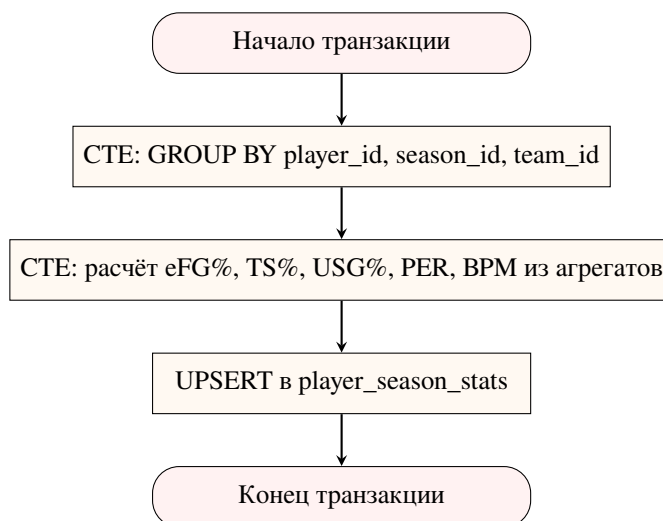


Рисунок 2.3 — Алгоритм обновления агрегатов

2.4 Проектирование ролевой модели доступа

В базе данных определены три роли: `db_admin`, `analyst` и `reader`. Администратору выданы полные права на таблицы, последовательности и функции схемы. Аналитику доступны выборка из таблиц и выполнение расчётных функций. Читатель работает исключительно через представления без прямого доступа к базовым таблицам. Создание и изменение структуры объектов базы данных выполняются от имени выделенного пользователя-владельца схемы при развёртывании. Ограничение доступа роли `reader` к полному представлению `v_player_rankings` и выдача прав только на `v_top_players` обусловлены бизнес-логикой: публичным пользователям доступна лишь краткая сводка лидеров, в то время как полные аналитические выгрузки предназначены только для аналитиков.

Таблица 2.9 — Матрица прав доступа к объектам базы данных

Объект	db_admin	analyst	reader
positions, seasons, teams, players	CRUD	SELECT	—
games, game_player_stats	CRUD	SELECT	—
player_season_stats, player_team_history	CRUD	SELECT	—
Функции calculate_*	EXECUTE	EXECUTE	—
v_team_stats, v_team_standings, v_top_players	—	SELECT	SELECT
v_player_rankings	SELECT	SELECT	—

2.5 Проектирование программного обеспечения и кэширования

2.5.1 Архитектура приложения

Программное обеспечение реализует трёхзвенную архитектуру. Уровень представления — веб-клиент, формирующий HTTP-запросы к серверу. Уровень приложения — REST API-сервер, выполняющий маршрутизацию запросов, проверку авторизации и вызов процедур СУБД. Уровень данных — реляционная СУБД и in-memory кэш, взаимодействие которых описано в подразделе 2.5.3.

API организован в пять групп маршрутов (players, teams, stats, league, admin), соответствующих вариантам использования из раздела 1.5.2. Итого спроектировано 21 бизнес-маршрут.

2.5.2 Спецификация REST API

Сезон в параметрах запроса передаётся целочисленным `season_id` (внешний ключ таблицы `seasons`); метка `label` (например, 2023–24) возвращается в ответах API для отображения в интерфейсе. Список сезонов клиент получает через `GET /league/seasons`.

Таблица 2.10 — Маршруты группы /players

Метод	Путь	Параметры	Вариант исп.
GET	/players	season_id (обяз.), position, team_id, search, limit, offset	Просмотр статистики
GET	/players/{player_id}	—	Просмотр статистики
GET	/players/{player_id}/stats	season_id (опц.)	Просмотр статистики
GET	/players/{player_id}/gamelog	season_id (обяз.)	Просмотр статистики
GET	/players/{player_id}/career	—	Просмотр статистики
GET	/players/{player_id}/teams	—	Просмотр статистики

Таблица 2.11 — Маршруты группы /teams

Метод	Путь	Параметры	Вариант исп.
GET	/teams	—	Просмотр статистики
GET	/teams/standings	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}	—	Просмотр статистики
GET	/teams/{team_id}/roster	season_id (обяз.)	Просмотр статистики
GET	/teams/{team_id}/games	season_id (опц.)	Просмотр статистики

Таблица 2.12 — Маршруты группы /stats

Метод	Путь	Параметры	Вариант исп.
GET	/stats/leaders	metric, season_id (обяз.), team_id, position, min_games, limit	Просмотр статистики
GET	/stats/advanced	season_id (обяз.)	Расчёт метрик
GET	/stats/scatter	season_id (обяз.)	Расчёт метрик
GET	/stats/boxscore/{game_id}	—	Просмотр статистики
GET	/stats/compare/players	p1_id, p2_id, season_id (обяз.)	Просмотр статистики

Параметр `metric` в маршруте `/stats/leaders` принимает одно из значений: `avg_pts`, `avg_reb`, `avg_ast`, `avg_stl`, `avg_blk`, `per`, `ts_pct`, `efg_pct`, `bpm` и `usg_pct`.

Таблица 2.13 — Маршруты группы /league

Метод	Путь	Параметры	Вариант исп.
GET	/league/seasons	—	Просмотр статистики
GET	/league/dashboard	season_id (обяз.)	Просмотр статистики
GET	/league/trends	—	Просмотр статистики
GET	/league/search	q (обяз., мин. 2 символа)	Просмотр статистики

Маршрут /league/seasons возвращает перечень доступных сезонов. Ответ содержит поля season_id, label, season_type, start_date и end_date; клиент использует его для формирования элементов выбора сезона. Маршрут /league/search реализует полнотекстовый поиск или поиск по подстроке (ILIKE) по полям first_name и last_name таблицы players, а также name таблицы teams, возвращая комбинированный массив совпадений.

Таблица 2.14 — Маршруты группы /admin

Метод	Путь	Параметры	Вариант исп.
POST	/admin/refresh/{season_id}	заголовок X-API-Key	Управление данными

Разграничение доступа на уровне API реализовано для группы /admin: маршрут проверяет наличие корректного значения в заголовке X-API-Key. Разграничение ролей analyst и reader обеспечивается средствами СУБД в соответствии с матрицей прав (таблица 2.9): API-сервер устанавливает соединение с базой данных от имени пользователя с соответствующим набором привилегий.

2.5.3 Кэширование результатов запросов

Для обеспечения высокой пропускной способности спроектирован паттерн Cache-Aside с использованием in-memory хранилища и API-сервера.

Алгоритм: приложению поступает REST-запрос. Сервер проверяет наличие ключа в кэше. При попадании данные возвращаются из кэша. При промахе сервер запрашивает реляционную БД, сериализует ответ, сохраняет его в кэш с заданным временем жизни и возвращает клиенту.

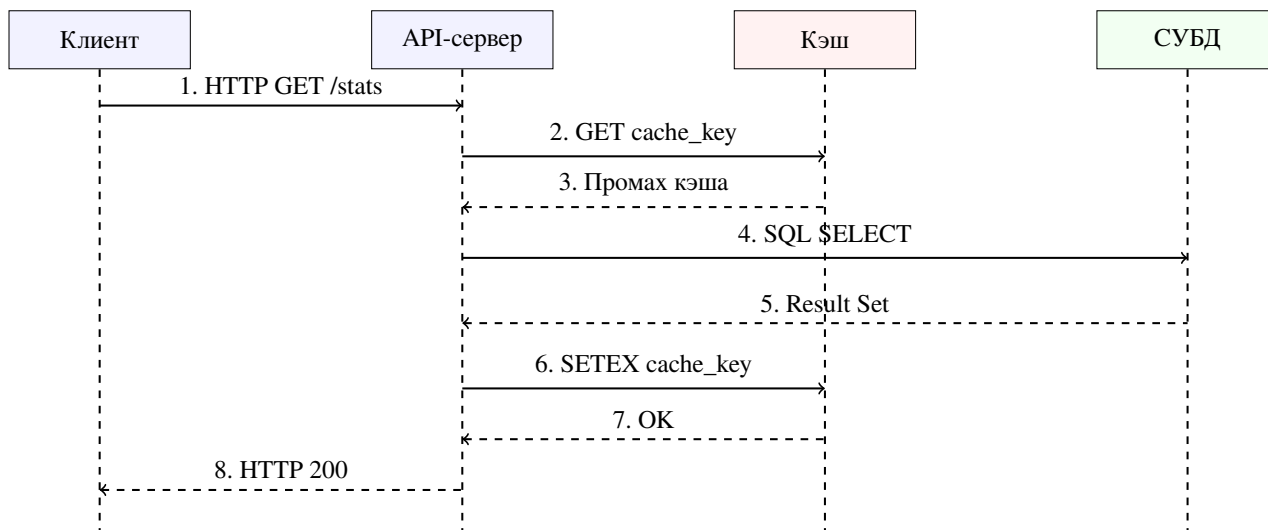


Рисунок 2.4 — UML-диаграмма последовательности при паттерне Cache-Aside

Инвалидация кэша выполняется по префиксам конкретных матчей и турнирных таблиц только для изменённого сезона, а не полным сбросом кэша: это предотвращает одновременную инвалидацию всех ключей и снижает риск пиковой нагрузки на СУБД при массовом промахе кэша — эффекта «шторма кэша», который возникает, когда множество параллельных запросов одновременно обнаруживают истёкший ключ и нагружают базу данных.

Вывод

Спроектирована реляционная схема из 8 таблиц, 4 представлений и 6 ключевых индексов; показано соответствие всех таблиц 1НФ, 2НФ и 3НФ. Бизнес-логика метрик реализована SQL-процедурами на стороне СУБД; ограничения целостности закреплены на уровне CHECK-ограничений и триггера. Определена ролевая модель из трёх ролей с разграничением прав на уровне таблиц, функций и представлений. Спроектировано REST API из 21 маршрута, организованных в пять групп и покрывающих все три варианта использования системы. Для снижения задержки аналитических выборок спроектировано кэширование по паттерну Cache-Aside с префиксной инвалидацией.

3 Технологический раздел

3.1 Выбор и обоснование средств реализации

Разрабатываемая система является дата-интенсивным приложением, в котором основная вычислительная нагрузка ложится на слой хранения данных. Выбор технологического стека обусловлен необходимостью поддержания строгой транзакционной целостности (ACID), выполнения сложных аналитических расчётов и обеспечения высокой скорости чтения с помощью кэширования.

Слой базы данных. В качестве основной СУБД выбрана PostgreSQL 15. В отличие от рассматриваемых альтернатив (MySQL 8, MS SQL Server), PostgreSQL является свободно распространяемой системой с мощным встроенным языком PL/pgSQL. Данный язык критически важен для реализации алгоритмов расчёта спортивных метрик непосредственно на стороне хранилища. Кроме того, PostgreSQL обладает нативной поддержкой конструкции UPSERT и расширенными возможностями проверки ограничений целостности (CHECK).

Слой кэширования. Для снижения нагрузки на дисковую подсистему при повторных аналитических запросах выбрана in-memory СУБД Redis 7. В отличие от Memcached, Redis поддерживает сложные структуры данных и позволяет выполнять эффективную инвалидацию кэша по префиксу ключа (pattern matching).

Серверный и клиентский слои. Программный интерфейс (REST API) реализован на языке Python с применением асинхронного фреймворка FastAPI и драйвера asyncpg, обеспечивающего высокопроизводительное неблокирующее взаимодействие с базой данных. Клиентская часть разработана в виде Single Page Application (SPA) на базе React/TypeScript, чья задача сводится к маршрутизации REST-запросов и визуализации полученной статистики.

Таблица 3.1 — Обоснование выбора технологий

Уровень	Выбрано	Альт. 1	Альт. 2	Основание выбора
СУБД	PostgreSQL 15	MySQL 8	MS SQL Server	Бесплатная лицензия, развитый PL/pgSQL, расширенные CHECK-ограничения и поддержка UPSERT [17]
Кэш	Redis 7	Memcached	Hazelcast	Высокая производительность, атомарные операции и инвалидация по префиксу ключей [16]
Backend	Python FastAPI	/ Django REST	Node.js Express	/ Асинхронная обработка (ASGI), нативная интеграция с высокоскоростным драйвером asynсpg
Frontend	React TypeScript	/ Vue 3	Angular	Строгая типизация моделей данных API; экосистема аналитических библиотек (Recharts)

3.2 Физическая реализация базы данных

3.2.1 Создание таблиц и ограничений

Физическая реализация структуры выполнена DDL-скриптами в соответствии со схемой, спроектированной в разделе 2.1. Полный код создания всех восьми таблиц, внешних ключей и ограничений вынесён в Приложение А.

В листинге 3.1 приведён фрагмент создания таблицы фактов `game_player_stats`, иллюстрирующий декларативное обеспечение ссылочной и логической целостности на уровне СУБД.

Листинг 3.1 — DDL таблицы `game_player_stats` (фрагмент)

```
CREATE TABLE game_player_stats (
    stat_id SERIAL PRIMARY KEY,
    game_id INTEGER NOT NULL REFERENCES games(game_id) ON DELETE
    CASCADE,
    player_id INTEGER NOT NULL REFERENCES players(player_id) ON
    DELETE RESTRICT,
    team_id INTEGER NOT NULL REFERENCES teams(team_id) ON DELETE
    RESTRICT,
    points SMALLINT NOT NULL DEFAULT 0 CHECK (points >= 0),
    fouls SMALLINT NOT NULL DEFAULT 0 CHECK (fouls BETWEEN 0
    AND 6),
    CONSTRAINT uq_game_player UNIQUE (game_id, player_id)
```

```
);
```

3.2.2 Реализация индексов

Все индексы, спроектированные для ускорения аналитических агрегаций, созданы на основе структуры В-дерева. Полный перечень индексов приведён в Приложении А. В листинге 3.2 продемонстрированы ключевые операторы создания индексов для таблицы фактов.

Листинг 3.2 — Создание индексов аналитических запросов (фрагмент)

```
CREATE INDEX idx_gps_player_id ON game_player_stats(player_id);
CREATE INDEX idx_gps_team_game ON game_player_stats(team_id,
game_id);
CREATE INDEX idx_pss_season ON player_season_stats(season_id, per
DESC NULLS LAST);
```

Исследование эффективности данных структур и их влияния на время выполнения запросов приведено в исследовательском разделе (глава 4).

3.2.3 Загрузка тестовых данных

Первоначальное наполнение базы данных выполнялось программным ETL-скриптом `load_data.py` на языке Python. Скрипт извлекает архивную статистику NBA за пять регулярных сезонов (2019/20–2023/24) из официального портала NBA Stats [1]. Доступ к протоколам матчей (Box Score) и справочникам лиги реализован через библиотеку `nba_api` [18], формирующую HTTP-запросы к сервису `stats.nba.com` и возвращающую структурированные ответы (JSON).

Полученные данные нормализуются в скрипте и загружаются в реляционные таблицы PostgreSQL пакетами (`batch insert`) с соблюдением порядка внешних ключей. Данный подход гарантирует происхождение данных (*data provenance*) из официального источника и сохраняет естественное распределение показателей реальных игроков. Естественное распределение критически важно для корректной верификации относительных метрик (таких как BPM и PER).

По результатам контрольных запросов объём таблицы фактов `game_player_stats` превысил 150 000 записей, а таблицы `games` — 5 800 матчей. Это полностью покрывает нефункциональное требование об объёме данных (рисунок 3.1).

Скриншот результата выполнения `SELECT COUNT(*) FROM game_player_stats` и `COUNT(*) FROM games`, подтверждающего наличие 156 314 и 5 842 записей соответственно.

Рисунок 3.1 — Подтверждение объёма данных в таблицах

3.3 Реализация логики предметной области в СУБД

3.3.1 Хранимые функции расчёта метрик

Все математические алгоритмы расчёта метрик перенесены на слой СУБД и реализованы в виде хранимых функций на языке PL/pgSQL. Исходный код всех функций вынесён в Приложение Б.

В листинге 3.3 приведён фрагмент функции `calculate_ts`, вычисляющей истинный процент бросков (TS%) с программной защитой от деления на ноль.

Листинг 3.3 — Реализация функции `calculate_ts` (фрагмент)

```
CREATE OR REPLACE FUNCTION calculate_ts(  
    p_player_id INT, p_season_id INT, p_team_id INT DEFAULT NULL  
) RETURNS DECIMAL(5,4) LANGUAGE plpgsql STABLE AS $$  
DECLARE  
    v_pts NUMERIC; v_fga NUMERIC; v_fta NUMERIC; v_denom NUMERIC;  
BEGIN  
    IF v_fga = 0 AND v_fta = 0 THEN RETURN NULL; END IF;  
  
    v_denom := 2.0 * (v_fga + 0.44 * v_fta);  
    IF v_denom = 0 THEN RETURN NULL; END IF;  
  
    RETURN ROUND(v_pts::DECIMAL / v_denom, 4);  
END; $$;
```

Все расчётные функции помечены квалификатором `STABLE`, что позволяет оптимизатору запросов кэшировать результаты вызовов в рамках одного оператора. Агрегация сезонной статистики выполняется хранимой процедурой `update_season_stats` с использованием атомарной конструкции `INSERT ... ON CONFLICT DO UPDATE`.

3.3.2 Триггеры логической целостности

Для защиты данных от аномалий реализован триггер `trg_check_team_score`. Он проверяет фундаментальное правило предметной области: сумма очков всех игроков команды в конкретном матче должна быть равна итоговому командному счёту.

Листинг 3.4 — Триггер проверки корректности очков (фрагмент)

```
CREATE OR REPLACE FUNCTION trg_validate_team_score_row()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_sum INT; v_expected INT;
BEGIN
    IF v_expected IS NOT NULL AND v_sum <> v_expected THEN
        RAISE EXCEPTION
            'Player points sum (%) does not match team score (%)',
            v_sum, v_expected;
    END IF;
    RETURN COALESCE(NEW, OLD);
END; $$;
```

3.4 Реализация ролевой модели доступа

Разграничение прав реализовано с использованием ролей базы данных (`db_admin`, `analyst`, `reader`). В соответствии с принципом минимальных привилегий, публичной роли `reader` запрещён прямой доступ к таблицам фактов; данные предоставляются исключительно через защищённые представления-витрины (листинг 3.5).

Листинг 3.5 — Разграничение доступа командными привилегиями

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO analyst;

GRANT SELECT ON v_team_stats TO reader;
GRANT SELECT ON v_top_players TO reader;
```

Проверка выполнена путём подключения к БД от имени роли `reader`: прямой запрос к `game_player_stats` завершается ожидаемой ошибкой `permission denied`.

3.5 Реализация программного обеспечения и интерфейса

3.5.1 Слой маршрутизации и взаимодействия с БД (Backend)

Серверная часть приложения выполняет роль прослойки доступа к данным (Data Access Layer). Бэкенд реализован на фреймворке FastAPI в соответствии со спецификацией REST API конструкторского раздела (подраздел 2.5.2). Для взаимодействия с PostgreSQL применяется асинхронный драйвер `asynspg` в связке с ORM `SQLAlchemy 2`. Движки подключений настроены с пулом соединений (Connection Pooling, `pool_size=10`, `max_overflow=20`), что позволяет СУБД обслуживать конкурентные запросы аналитиков без постоянного открытия новых TCP-сессий.

Жизненный цикл сессии базы данных управляется через механизм внедрения зависимостей (Dependency Injection): для каждого HTTP-запроса создаётся экземпляр `AsyncSession`, привязанный к роли пользователя СУБД (`analyst`, `reader`, `db_admin`), и гарантированно закрывается по завершении обработки маршрута. Для снижения нагрузки на дисковую подсистему реализован паттерн `Cache-Aside` на базе Redis (класс `CacheManager`). В листинге 3.6 показан фрагмент маршрута `GET /players/{player_id}/stats`; полный исходный код вынесён в приложение В.

Листинг 3.6 — Реализация взаимодействия с БД и кэшем (фрагмент)

```
@router.get("/{player_id}/stats")
async def get_player_stats(
    player_id: int,
    season_id: int | None = None,
    cache: CacheManager = Depends(get_cache),
    db: AsyncSession = Depends(get_db_analyst),
):
    cache_key = f"player_stats:{player_id}:{season_id or 'all'}"
    cached = await cache.get(cache_key)
    if cached:
        return cached

    query = (
        select(PlayerSeasonStats.season_id, PlayerSeasonStats.
ts_pct, ...)
        .where(PlayerSeasonStats.player_id == player_id)
    )
    if season_id:
        query = query.where(PlayerSeasonStats.season_id ==
season_id)
```

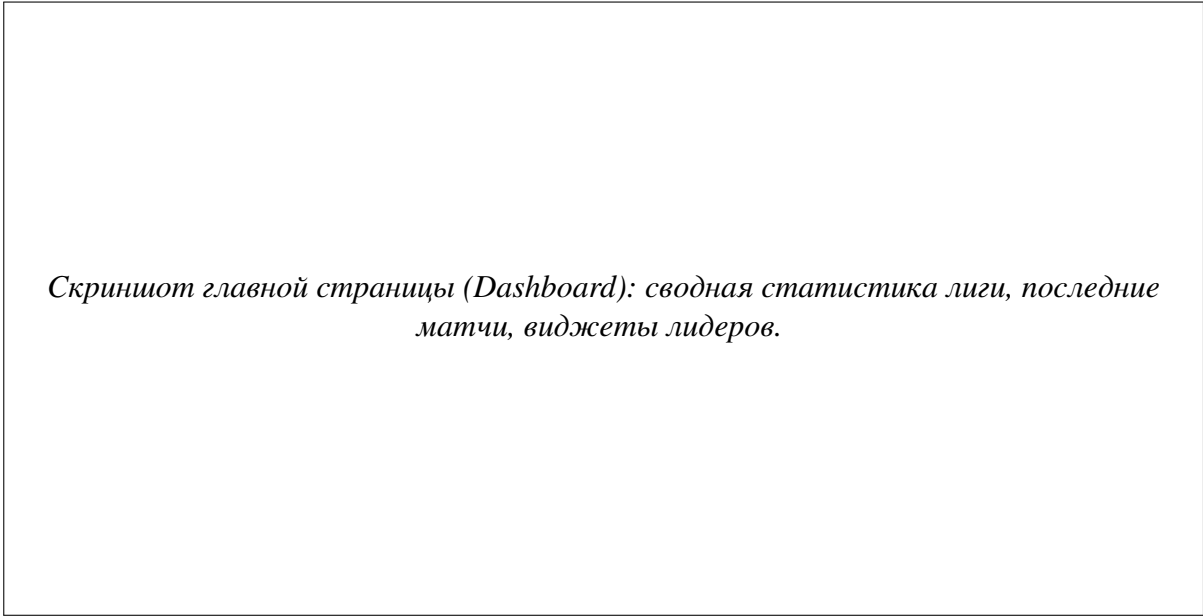
```
result = await db.execute(query)
data = [dict(row) for row in result.mappings().all()]
await cache.set(cache_key, data, ttl=300)
return data
```

3.5.2 Пользовательский интерфейс (Frontend)

Клиентское веб-приложение реализовано на React (TypeScript) и предоставляет графический интерфейс для анализа данных, извлекаемых из СУБД через REST API. Приложение не дублирует бизнес-логику расчёта метрик — все агрегаты формируются на стороне PostgreSQL (функции, представления, витрины). Основные разделы интерфейса:

- 1) **Dashboard** — сводная статистика лиги, последние матчи и ключевые показатели сезона (данные из представлений и агрегатов СУБД);
- 2) **Players / Teams** — справочники с серверным поиском (маршрут `/league/search`, индекс `idx_players_name`) и фильтрацией по сезону и команде;
- 3) **Leaderboard / Standings** — рейтинги лидеров и турнирная таблица по метрикам PTS, AST, REB, PER, TS%, eFG%, BPM;
- 4) **Compare** — интерактивное сравнение двух игроков (маршрут `/stats/compare`).

Метрики визуализируются радарными и столбчатыми диаграммами библиотеки Recharts. На рисунке 3.2 представлен внешний вид главной страницы (Dashboard).



Скриншот главной страницы (Dashboard): сводная статистика лиги, последние матчи, виджеты лидеров.

Рисунок 3.2 — Главная страница приложения (Dashboard)

На рисунке 3.3 показан интерфейс раздела сравнения профилей игроков, демонстрирующий выдачу сложных агрегатов из базы данных.

Скриншот страницы сравнения двух игроков (Compare): радарный график продвинутых метрик, столбчатые диаграммы базовой статистики.

Рисунок 3.3 — Страница сравнения игроков

3.6 Тестирование и верификация аналитических данных

Тестирование логики СУБД проводилось методом чёрного ящика. Для проверки точности математических агрегаций выполнена ручная верификация на реальных данных из загруженной базы.

В качестве тестового объекта был выбран профиль игрока Nikola Jokić (`player_id = 2327`) за регулярный сезон 2023-24 (`season_id = 5`). Прямой SQL-запрос суммы сырых фактов из таблицы `game_player_stats` вернул следующие значения:

- очки (PTS): 2085;
- броски с игры (FGA): 1411;
- штрафные попытки (FTA): 438.

Ручной расчёт истинного процента бросков (TS%) по формуле (1.2):

$$TS\% = \frac{2085}{2 \cdot (1411 + 0,44 \cdot 438)} = \frac{2085}{2 \cdot 1603,72} = \frac{2085}{3207,44} \approx 0,6501.$$

Для верификации в СУБД был выполнен вызов разработанной хранимой функции:

```
SELECT calculate_ts(2327, 5);
```

Результат, возвращённый базой данных, абсолютно совпал с ручным аналитическим расчётом. Аналогичная верификация проведена для метрик `eFG%` и `USG%`; расхождений не выявлено. Тестирование граничных случаев (отсутствие попыток бросков, нулевое время на площадке) подтвердило корректную обработку исключений и возврат значения `NULL` без прерывания транзакций (таблица 3.2).

Таблица 3.2 — Контрольные тест-кейсы функций расчёта метрик

№	Сценарий	Входные данные	Ожидание	Результат
1	Нормальный профиль	<code>calculate_ts(2327, 5)</code>	$\approx 0,6501$	совпало
2	Нет попыток броска	<code>calculate_ts(0, 0)</code>	NULL	NULL
3	Нулевые FGA (eFG%)	<code>calculate_efg(0, 0)</code>	NULL	NULL
4	Пересчёт витрины	<code>CALL update_season_stats(5)</code>	обновлены стро- ки pss	выполнено

Вывод

В технологическом разделе обоснован программный стек: PostgreSQL, Redis, FastAPI и React. Реализована физическая реляционная схема базы данных, включающая ссылочные ограничения и структуры В-деревьев. Разработанный ETL-скрипт обеспечил загрузку более 150 тысяч реальных фактов статистики из открытых источников. Вся расчётная логика предметной области успешно перенесена на уровень хранилища в виде функций PL/pgSQL и триггеров целостности. Реализована ролевая модель доступа, серверное REST API с пулом соединений и кэшированием Cache-Aside, а также клиентский интерфейс на React. Проведённая верификация с ручным расчётом метрик на живых данных подтвердила полную математическую корректность функционирования базы данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. NBA Stats [Электронный ресурс]. — Официальная статистика Национальной баскетбольной ассоциации. — Режим доступа: <https://www.nba.com/stats> (дата обращения: 29.03.2026).
2. Kubatko, J. et al. A Starting Point for Analyzing Basketball Statistics // Journal of Quantitative Analysis in Sports. — 2007. — Vol. 3, № 3. — Article 1.
3. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг ; пер. с англ. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
4. Basketball-Reference.com [Электронный ресурс]. — Sports Reference LLC. Справочник статистики NBA и исторические таблицы. — Режим доступа: <https://www.basketball-reference.com/> (дата обращения: 29.03.2026).
5. ESPN NBA [Электронный ресурс]. — Новости и обзоры НБА. — Режим доступа: <https://www.espn.com/nba/> (дата обращения: 29.03.2026).
6. Hollinger, J. Pro Basketball Forecast. — Washington, D.C.: Potomac Books, 2005. — 432 p.
7. Player Efficiency Rating [Электронный ресурс] // Basketball-Reference.com. — Описание показателя PER (упрощенная методология). — Режим доступа: <https://www.basketball-reference.com/about/per.html> (дата обращения: 29.03.2026).
8. Myers, D. About Box Plus/Minus (BPM) 2.0 [Электронный ресурс] // Basketball-Reference.com. — Режим доступа: <https://www.basketball-reference.com/about/bpm2.html> (дата обращения: 29.03.2026).
9. Glossary — NBA.com : Stats [Электронный ресурс]. — Термины и определения показателей. — Режим доступа: <https://www.nba.com/stats/help/glossary> (дата обращения: 29.03.2026).
10. Oliver, D. Basketball on Paper: Rules and Tools for Performance Analysis. — Washington, D.C.: Potomac Books, 2004. — 378 p.
11. Chen, P. P. The Entity-Relationship Model — Toward a Unified View of Data / P. P. Chen // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.
12. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — СПб.: Питер, 2018. — 640 с.

13. Садаладж, П. Дж. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — М.: Вильямс, 2013. — 192 с.
14. ClickHouse Documentation [Электронный ресурс]. — Официальная документация СУБД ClickHouse. — Режим доступа: <https://clickhouse.com/docs/> (дата обращения: 29.03.2026).
15. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт ; пер. с англ. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
16. Redis Documentation [Электронный ресурс]. — Документация in-memory хранилища Redis. — Режим доступа: <https://redis.io/docs/> (дата обращения: 29.03.2026).
17. PostgreSQL 15 Documentation [Электронный ресурс]. — Официальная документация СУБД PostgreSQL. — Режим доступа: <https://www.postgresql.org/docs/15/> (дата обращения: 29.03.2026).
18. nba_api: an API Client Package to Access the APIs of NBA.com [Электронный ресурс]. — Документация Python-библиотеки для доступа к stats.nba.com. — Режим доступа: https://github.com/swar/nba_api (дата обращения: 29.03.2026).

ПРИЛОЖЕНИЯ

Приложение А. Скрипты создания структуры базы данных

```
CREATE TABLE positions (  
    position_id SERIAL PRIMARY KEY,  
    code VARCHAR(2) UNIQUE NOT NULL,  
    name VARCHAR(20) NOT NULL  
);  
  
CREATE TABLE seasons (  
    season_id SERIAL PRIMARY KEY,  
    label VARCHAR(9) UNIQUE NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    season_type VARCHAR(15) NOT NULL  
);  
  
CREATE TABLE teams (  
    team_id SERIAL PRIMARY KEY,  
    nba_team_id INT UNIQUE NOT NULL,  
    name VARCHAR(60) UNIQUE NOT NULL,  
    abbreviation CHAR(3) UNIQUE NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    conference VARCHAR(4) NOT NULL,  
    division VARCHAR(20) NOT NULL,  
    arena_name VARCHAR(80),  
    founded_year SMALLINT,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

Приложение Б. Скрипты создания функций и процедур

```
CREATE OR REPLACE FUNCTION calculate_efg(  
    p_player_id INT,  
    p_season_id INT,  
    p_team_id INT DEFAULT NULL  
)  
RETURNS DECIMAL(5,4)  
LANGUAGE plpgsql  
STABLE
```

```

AS $$
DECLARE
    v_fgm NUMERIC := 0;
    v_fg3m NUMERIC := 0;
    v_fga NUMERIC := 0;
BEGIN
    SELECT
        COALESCE(SUM(gps.fgm), 0),
        COALESCE(SUM(gps.fg3m), 0),
        COALESCE(SUM(gps.fga), 0)
    INTO v_fgm, v_fg3m, v_fga
    FROM game_player_stats gps
    JOIN games g ON g.game_id = gps.game_id
    WHERE gps.player_id = p_player_id
        AND g.season_id = p_season_id
        AND (p_team_id IS NULL OR gps.team_id = p_team_id);

    IF v_fga = 0 THEN
        RETURN NULL;
    END IF;

    RETURN ROUND((v_fgm + 0.5 * v_fg3m) / v_fga, 4);
END;
$$;

```

Приложение В. Исходный код механизма кэширования

```

import json
from functools import wraps
from fastapi import Request, Response

def cache_response(ttl_seconds: int = 300):
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            request: Request = kwargs.get("request")
            redis_client = request.app.state.redis

            cache_key = f"api_cache:{request.url.path}?{request.url
                .query}"

```

```
        cached_data = await redis_client.get(cache_key)
        if cached_data:
            return Response(
                content=cached_data,
                media_type="application/json"
            )

        response = await func(*args, **kwargs)

        await redis_client.setex(
            cache_key,
            ttl_seconds,
            json.dumps(response)
        )
        return response
    return wrapper
return decorator
```